

8.7.1.2 Paginación de Resultados a Nivel de Base de Datos con Hibernate, JPA y Spring Data

Resumen

Esta lectura teórica examina la implementación eficiente de **paginación y limitación de resultados** a nivel de base de datos en aplicaciones Java desarrolladas con **Hibernate, Spring Data JPA y Spring Web**. Usted aprenderá a prevenir el colapso de memoria por carga masiva de datos en el servidor (*In-Memory Pagination*), a utilizar las abstracciones `Pageable`, `PageRequest`, `Page<T>`, `Slice<T>` y `Sort`, a analizar las cláusulas SQL emitidas (`OFFSET / FETCH NEXT` y `LIMIT`), y a estructurar respuestas JSON estandarizadas en controladores RESTful integrados con capas cliente.

Concepto Principal (El 'Qué')

En aplicaciones empresariales con tablas que albergan cientos de miles o millones de registros, retornar la totalidad de los datos en una sola consulta HTTP produce un impacto catastrófico en el rendimiento: sobrecarga de red, alto consumo de memoria RAM en la JVM y tiempos de respuesta inaceptables.

La **paginación a nivel de base de datos** resuelve esta problemática al delegar al motor relacional la tarea de recortar y devolver únicamente la sub-cobertura o "página" de registros solicitada por el usuario.

1. La mecánica subyacente en SQL e Hibernate

Hibernate traduce los comandos de paginación en cláusulas SQL nativas mediante los métodos de su API de consultas: * `query.setFirstResult(int startPosition)`: Define el desplazador u desplazamiento inicial (*OFFSET*). * `query.setMaxResults(int maxResults)`: Define el límite

máximo de filas a retornar (*LIMIT* o *FETCH NEXT*).

Por ejemplo, en Microsoft SQL Server Hibernate genera:

```
SELECT * FROM Producto ORDER BY id OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

En PostgreSQL o MySQL Hibernate genera:

```
SELECT * FROM Producto ORDER BY id LIMIT 10 OFFSET 20;
```

2. Abstracciones clave de Spring Data JPA

- **Pageable:** Interfaz que representa la solicitud de paginación (número de página `page`, tamaño de página `size` y criterios de ordenamiento `Sort`). Su implementación concreta más común es `PageRequest.of(page, size, Sort.by(...))`. Cabe destacar que en Spring Data los índices de página inician en **0** (`page = 0` es la primera página).
- **Page<T>:** Sub-cobertura de resultados que contiene la lista de elementos de la página actual y **metadatos globales de la consulta**: total de elementos (`totalElements`), total de páginas (`totalPages`), página actual (`number`), etc. Para calcular `totalElements`, Spring Data ejecuta automáticamente una consulta adicional de conteo (`SELECT COUNT(*)`).
- **Slice<T>:** Alternativa optimizada a `Page<T>`. Retorna los elementos de la página actual e indica únicamente si existe una página posterior (`hasNext()`), **sin ejecutar la consulta adicional de conteo `COUNT(*)`**. Logra esto consultando internamente `size + 1` registros.
- **Sort:** Estructura que define la columna o lista de propiedades por las cuales ordenar los resultados, así como la dirección de ordenamiento (`Sort.Direction.ASC` o `Sort.Direction.DESC`).

Analogía (La Intuición)

Imagine que usted visita una biblioteca nacional que posee un catálogo de **1.000.000 de libros**.

- **Sin Paginación (Carga Total):** Usted pide al bibliotecario que le traiga los libros. El bibliotecario carga 1.000.000 de tomos en montacargas, colapsando el pasillo y aplastando

su mesa de lectura (la memoria RAM de la JVM se agota con un `OutOfMemoryError`).

- **Paginación en Memoria (In-Memory Pagination):** El bibliotecario le trae los 1.000.000 de libros a su mesa, pero usted solo abre la caja que contiene del libro #21 al #30. Aunque usted solo leyó 10 libros, el esfuerzo de transporte fue destructivo.
- **Paginación a Nivel de Base de Datos (Pageable):** Usted solicita explícitamente: *"Tráigame únicamente 10 libros, comenzando desde el puesto 21"*. El bibliotecario camina directamente al estante, toma solo los 10 tomos exactos y se los entrega en una bandeja ligera. El tiempo de respuesta es casi instantáneo y la mesa permanece libre.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se ilustra la implementación de un endpoint RESTful paginado con DTOs en Spring Boot 3.x.

1. Entidad y DTO de Respuesta

```
package cr.ac.ucr.ie.domain;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "Producto")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 150)
    private String nombre;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal precio;

    @Column(nullable = false)
    private Integer stock;
```

```

    @Column(nullable = false, length = 50)
    private String categoria;

    public Producto() {}

    public Producto(String nombre, BigDecimal precio, Integer stock, String categoria) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
        this.categoria = categoria;
    }

    // Getters y Setters
    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public BigDecimal getPrecio() { return precio; }
    public Integer getStock() { return stock; }
    public String getCategoria() { return categoria; }
}

```

```

package cr.ac.ucr.ie.dto;

import java.math.BigDecimal;

public record ProductoDTO(
    Long id,
    String nombre,
    BigDecimal precio,
    Integer stock,
    String categoria
) {}

```

2. Repositorio con Soporte para Pageable y Slice

```

package cr.ac.ucr.ie.data;

import cr.ac.ucr.ie.domain.Producto;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Slice;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

```

```

import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {

    // Paginación derivativa automática basada en convención de nombre
    Page<Producto> findByCategoria(String categoria, Pageable pageable);

    // Consulta JPQL personalizada con paginación
    @Query("SELECT p FROM Producto p WHERE p.precio >= :precioMinimo")
    Page<Producto> buscarPorPrecioMinimo(@Param("precioMinimo") Double precioMinimo, Pageable pageable);

    // Optimización con Slice (Sin consulta de conteo SELECT COUNT(*))
    Slice<Producto> findByNombreContainingIgnoreCase(String termino, Pageable pageable);
}

```

3. Servicio de Negocio y Transformación a DTOs

```

package cr.ac.ucr.ie.business;

import cr.ac.ucr.ie.data.ProductoRepository;
import cr.ac.ucr.ie.domain.Producto;
import cr.ac.ucr.ie.dto.ProductoDTO;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ProductoService {

    private final ProductoRepository productoRepository;

    public ProductoService(ProductoRepository productoRepository) {
        this.productoRepository = productoRepository;
    }

    @Transactional(readOnly = true)
    public Page<ProductoDTO> obtenerProductosPaginados(int pagina, int tamano, String campoOrden, Sort.Direction dir = direction.equalsIgnoreCase("DESC") ? Sort.Direction.DESC : Sort.Direction.ASC, Pageable pageable = PageRequest.of(pagina, tamano, Sort.by(dir, campoOrden)));
}

```

```

    Page<Producto> entidadPage = productoRepository.findAll(pageable);

    // Mapeo transparente de Entidad a DTO conservando metadatos de paginación
    return entidadPage.map(p -> new ProductoDTO(
        p.getId(),
        p.getNombre(),
        p.getPrecio(),
        p.getStock(),
        p.getCategoria()
    ));
}
}

```

4. Controlador RESTful (@RestController)

```

package cr.ac.ucr.ie.controller;

import cr.ac.ucr.ie.business.ProductoService;
import cr.ac.ucr.ie.dto.ProductoDTO;
import org.springframework.data.domain.Page;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/productos")
@CrossOrigin(origins = "*")
public class ProductoController {

    private final ProductoService productoService;

    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping
    public ResponseEntity<Page<ProductoDTO>> listarProductos(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "10") int size,
        @RequestParam(defaultValue = "id") String sortBy,
        @RequestParam(defaultValue = "ASC") String direction) {

        Page<ProductoDTO> productosPaginados = productoService.obtenerProductosPaginados(page, size, sortBy, direction);
        return ResponseEntity.ok(productosPaginados);
    }
}

```

```
}  
}
```

Trampas Comunes

1. Paginación en Memoria con `JOIN FETCH` y `@OneToMany` (Advertencia HHH000104)

Al intentar paginar una consulta JPQL que incluye `FETCH JOIN` sobre una colección `@OneToMany` (ej. `SELECT c FROM Cliente c JOIN FETCH c.facturas`), Hibernate no puede aplicar `LIMIT / OFFSET` en la SQL resultante sin duplicar registros principales. Por lo tanto, Hibernate carga **toda la base de datos en memoria RAM** y emite la advertencia: `HHH000104: firstResult/maxResults specified with collection fetch; applying in-memory!`. * **Solución:** Eliminar el `JOIN FETCH` y utilizar `@EntityGraph(attributePaths = {"facturas"})` o paginar en dos pasos.

2. Cuello de Botella del `SELECT COUNT(*)` en Tablas Gigantescas

Para retornar un objeto `Page<T>`, Spring Data debe contar el total absoluto de registros ejecutando una segunda consulta `SELECT COUNT(*)`. En tablas de decenas de millones de filas sin índices adecuados, la consulta de conteo puede demorar varios segundos. * **Solución:** Reemplazar `Page<T>` por `Slice<T>` o proporcionar un `countQuery` optimizado en la anotación `@Query`.

3. Confusión con el Índice Base Cero de `Pageable`

Los desarrolladores o clientes de UI frecuentemente envían `page = 1` esperando la primera página. Al pasarlo directamente a `PageRequest.of(1, 10)`, Spring Data retornará la **segunda página** (registros 11 al 20). Siempre asegúrese de restar 1 en el controlador si la API expone paginación basada en 1.

Pregunta de Reflexión

Si su aplicación expone un buscador de productos donde los usuarios navegan con scroll infinito (como en redes sociales) cargando 20 nuevos elementos al deslizar hacia abajo, ¿cuál abstracción

de Spring Data JPA (`Page<T>` o `Slice<T>`) resulta más eficiente en términos de uso de CPU y latencia de base de datos? Argumente su respuesta analizando el costo computacional de las consultas ejecutadas.